

ASSIGNMENT: 12

Name: Rajdeep Das

Roll No: CSB25203

Subject: Advanced Programming.

```
# =====
```

```
# E-Commerce Order Processing System using SOLID Principles
```

```
# =====
```

```
from abc import ABC, abstractmethod
```

```
# =====
```

```
# 1. PAYMENT MODULE
```

```
# SRP + OCP + LSP + ISP
```

```
# =====
```

```
class PaymentMethod(ABC):
```

```
    @abstractmethod def
```

```
    pay(self, amount):
```

```
        pass
```

```
class CreditCardPayment(PaymentMethod):
```

```
    def pay(self, amount):
```

```
        print("[Payment] Paid", amount, "using Credit Card")
```

```
class UPIPayment(PaymentMethod):  
    def pay(self, amount):  
        print("[Payment] Paid", amount, "using UPI")
```

```
class WalletPayment(PaymentMethod):  
    def pay(self, amount):  
        print("[Payment] Paid", amount, "using Wallet")
```

```
# =====  
# 2. NOTIFICATION MODULE  
# SRP + OCP + LSP + ISP  
# =====
```

```
class NotificationService(ABC):  
    @abstractmethod  
    def send(self, message):  
        pass
```

```
class EmailNotification(NotificationService):  
    def send(self, message):  
        print("[Email Notification]:", message)
```

```
class SMSNotification(NotificationService):  
    def send(self, message):  
        print("[SMS Notification]:", message)
```

```
class PushNotification(NotificationService):
    def send(self, message):
        print("[Push Notification]:", message)
```

```
# =====
```

```
# 3. STORAGE MODULE
```

```
# SRP + OCP + DIP
```

```
# =====
```

```
class Storage(ABC):
    @abstractmethod
    def save(self, order):
        pass
```

```
class DatabaseStorage(Storage):
    def save(self, order):
        print("[Database] Order saved:", order)
```

```
class FileStorage(Storage):
    def save(self, order):
        print("[File] Order saved:", order)
```

```
# =====
```

```
# 4. ORDER MODULE
```

```
# SRP + LSP
```

```

# =====
class Order(ABC):
    def __init__(self, order_id, customer_name, amount):
        self.order_id = order_id
        self.customer_name = customer_name
        self.amount = amount

    @abstractmethod
    def get_total(self):
        pass

    def __str__(self):
        return (
            "OrderID=" + str(self.order_id) +
            ", Customer=" + self.customer_name +
            ", Total=" + str(self.get_total())
        )

class RegularOrder(Order):
    def get_total(self):
        return self.amount

class DiscountedOrder(Order):
    def __init__(self, order_id, customer_name, amount, discount):
        Order.__init__(self, order_id, customer_name, amount)
        self.discount = discount

    def get_total(self):
        return self.amount - self.discount

```

```

class PriorityOrder(Order):
    def __init__(self, order_id,
customer_name, amount, priority_fee):
        Order.__init__(self,
order_id, customer_name, amount)
        self.priority_fee =
priority_fee

```

```

def get_total(self):
    return self.amount + self.priority_fee

```

```

# =====
# 5. ORDER SERVICE
# DIP: Depends on abstractions only
# =====

```

```

class OrderService:
    def __init__(self, payment_method,
notification_service, storage):
        self.payment_method = payment_method
self.notification_service = notification_service
self.storage
= storage

```

```

def place_order(self, order):

```

```

    total_amount = order.get_total()

```

```

    print("\n===== ORDER PROCESSING =====")

```

```

    # Process Payment

```

```

    self.payment_method.pay(total_amount)

```

```

        # Save Order
        self.storage.save(order)

        # Send Notification
        self.notification_service.send(
            "Order " + str(order.order_id) + " placed successfully."
        )

        print("=====")

# =====
# MAIN PROGRAM
# =====

if __name__ == "__main__":

    # -----
    # Example 1: Regular Order
    # -----

    order1 = RegularOrder(101, "Rajdeep", 5000)

    payment1 = CreditCardPayment()
    notification1 = EmailNotification()
    storage1 = DatabaseStorage()

    service1 = OrderService(payment1, notification1, storage1)

    service1.place_order(order1)

```

```
# -----
```

```
# Example 2: Discounted Order
```

```
# -----
```

```
order2 = DiscountedOrder(102, "Aman", 4000, 500)
```

```
payment2 = UPIPayment()
```

```
notification2 = SMSNotification()
```

```
storage2 = FileStorage()
```

```
service2 = OrderService(payment2, notification2, storage2)
```

```
service2.place_order(order2)
```

```
# -----
```

```
# Example 3: Priority Order
```

```
# -----
```

```
order3 = PriorityOrder(103, "Riya", 7000, 300)
```

```
payment3 = WalletPayment()
```

```
notification3 = PushNotification()
```

```
storage3 = DatabaseStorage()
```

```
service3 = OrderService(payment3, notification3, storage3)
```

```
service3.place_order(order3)
```